

Security Engineer Multi-Engine Pack - Beginner's Build Report (P1 -> P7)

Product: Sentinel Stacks

Role agent: Security Engineer

Tool ID: scan_security_engineer_pack

Main file: ai_security_engineer_pack.py

Final version: 0.7.0-p7

Status: Hands COMPLETE (10 / 10 engines active) + Remediation mapped (53 / 53 findings)

Lab repo: ibkbalo/secops-pipeline-lab

Report date: 2026-08-04

1. Start here - what is this?

Imagine you hire a Security Engineer to check whether your company's app and perimeter are safe. That person does not run just one tool. They look at network exposure, leaked files, API mistakes, login weaknesses, email spoofing, and more.

Sentinel Stacks built that job as software: the Security Engineer Hands Pack.

Plain English	Technical name
One security check	A single scanner (e.g. port scan onl
A full Security Engineer review	Multi-engine pack - 10 specialized c
The written result	TOOL_STANDARDS JSON report - machine
A fix kit for each problem	Remediation Sentinel - Terraform, co

What this phase achieved: We went from an empty shell (P1) to a complete 10-engine pack (P6) and then wired every finding to an automatic fix map (P7). You can demo the whole flow offline - no live target required.

2. Why we built it this way

2.1 The problem with "one scanner" demos

Many security projects stop at a single script: "we scan ports" or "we find secrets." Real Security Engineers cover many surfaces. Our pack follows the same pattern as the DevSecOps and Cloud packs already in this repo:

1. Hands first - deterministic scanners that always produce the same JSON shape.
2. Brain later - an AI agent that reads findings and decides next steps.
3. Face last - a GUI dashboard (not built yet).

This report covers Hands + Remediation mapping only.

2.2 Design rules (simple version)

Rule	What it means for you
------	-----------------------

Enterprise bar	We keep adding engines until the rol
Offline mock mode	Run python ai_security_engineer_pack
Stable IDs	Every issue has a permanent ID like
Optional live tools	If you install nmap, nuclei, etc., t

3. The 10 engines - what each one checks

Think of each engine as a specialist on the Security Engineer team:

Engine key	Code	Beginner question it answers
network	NET	Are dangerous ports open? Is TLS old
data_exposure	DATA	Are sensitive paths (.env, .git, bac
api	API	Are admin or debug APIs open without
vuln	VULN	Missing security headers? XSS, open
identity	IDENT	Session fixation? OAuth state skippe
governance	GOV	Missing security.txt? Server banner
phishing	PHISH	Can attackers spoof email (SPF/DKIM/
traffic	TRF	Unusual traffic patterns, bot abuse,
protocol	PRT	Weak or misconfigured protocols on t
asset	AST	Shadow assets, forgotten subdomains,

Finding ID format (locked forever):

```
PERIM-{ENGINE_CODE}-{NNN}
```

Examples: PERIM-PHISH-003, PERIM-API-007, PERIM-VULN-002.

4. How it fits together (architecture)

You run the pack

```

|
v
ai_security_engineer_pack.py
|
+-- Reads mock fixture OR live target path
+-- Runs 10 engine workers (NET, DATA, API, ?)
+-- Merges findings + severity scores
+-- Outputs TOOL_STANDARDS JSON
|
v
ai_remediation_engine.py (P7)
|
+-- Looks up each PERIM-* ID in FIX_MAP
+-- Generates Terraform / nginx conf / runbooks
+-- Zips a hardening kit (dry_run mode)
```

Key files:

File	Purpose
ai_security_engineer_pack.py	The pack - all 10 engines
mock_security_engineer_vulnerable.js	Fake "bad" company - 53 findings
mock_security_engineer_clean.json	Fake "good" company - 0 findings
tool_registry.json	Catalog entry for orchestrators
ai_remediation_engine.py	Turns findings into fix kits (v1.4.0)

5. Phase-by-phase - what we built

We delivered in 7 phases (P1 -> P7). Each phase turned on more engines and raised the finding count on the vulnerable mock.

5.1 Rollup table

Phase	Version	What turned on	Active engines	Vuln mock findings
P1	skeleton	Registry + fixtures only	0 / 10	0
P2	0.2.0-p2	Phishing (PHISH)	1 / 10	13
P3	0.3.0-p3	+ Network (NET), Data exposure (DATA)	3 / 10	25
P4	0.4.0-p4	+ API, Vulnerability (VULN)	5 / 10	38
P5	0.5.0-p5	+ Identity (IDENT), Governance (GOV)	7 / 10	45
P6	0.6.0-p6	+ Traffic (TRF), Protocol (PRT), As	10 / 10	53
P7	0.7.0-p7	FIX_MAP for all 53 PERIM-* IDs	10 / 10	53 mapped -> 0 unmapped

Pack completion: pack_hands_complete: true at P6. Remediation bar closed at P7.

5.2 P1 - Pack skeleton

Goal: Define the shape of the product before real checks.

- * Created ai_security_engineer_pack.py with engine registry, ID scheme, backend detection, JSON merge runner.
- * Added mock fixture files (vulnerable + clean profiles).
- * Registered scan_security_engineer_pack in tool_registry.json.
- * All engines started as stubs (registered but inactive) - 0 findings by design.

Why it mattered: Later phases only *activate* engines; they do not rewrite the whole pack.

5.3 P2 - Phishing engine

Goal: Can someone spoof email from our domain?

- * Activated PHISH engine (13 findings on vulnerable mock).
- * Checks DMARC policy, SPF, DKIM, and analyzes sample suspicious emails (BEC, credential harvest patterns).

5.4 P3 - Network + data exposure

Goal: Is the perimeter and data surface leaking?

- * NET (6): Old TLS, expired cert, risky open ports (SSH, MySQL, etc.), missing CDN/WAF.
- * DATA (6): Exposed .env, .git, backup paths, public S3-style buckets.

5.5 P4 - API + vulnerability

Goal: Are APIs and web layers weak?

- * API (7): OpenAPI exposed, unauthenticated admin/export endpoints, no rate limiting.
- * VULN (6): Missing CSP/HSTS/X-Frame-Options, XSS, open redirect, bad cookie flags.

5.6 P5 - Identity + governance

Goal: Are auth and public security posture solid?

- * IDENT (3): Session fixation risk, OAuth state not required, MFA not enforced.
- * GOV (4): No security.txt, no privacy policy link, server banner leak, weak HSTS.

5.7 P6 - Traffic + protocol + asset (hands complete)

Goal: Close the last gaps and hit 100% pack completion.

- * TRF (3): Edge traffic / abuse signals.
- * PRT (3): Protocol-level weaknesses (optional nmap backend when installed).
- * AST (2): Asset inventory / shadow surface gaps.

Final vulnerable mock breakdown (53 total):

Code	Count	Theme
PHISH	13	Email authentication + social engine
API	7	Shadow APIs, missing auth, no rate l
DATA	6	Leaked paths and public storage
NET	6	TLS, ports, WAF
VULN	6	Headers, XSS, cookies
GOV	4	Public security hygiene
IDENT	3	Login/session weaknesses
TRF	3	Traffic edge issues
PRT	3	Protocol misconfigurations
AST	2	Asset visibility gaps

Clean mock: 0 findings, all 10 engines report passed checks.

5.8 P7 - Remediation mapping (fix kits)

Goal: Every PERIM-* finding gets a known fix - not just a description.

- * Bumped ai_remediation_engine.py to v1.4.0.
- * Added 53 FIX_MAP entries for all Security Engineer pack IDs.

- * New config templates for perimeter fixes (WAF, DMARC, SPF, DKIM, mail, SIEM, session, OAuth, asset).
- * Smoke test: pack mock -> remediation engine -> 53 mapped, 0 unmapped, hardening kit ZIP generated.

What you get after P7: Scan -> structured JSON -> downloadable kit with Terraform snippets, nginx-style configs, and YAML runbooks.

6. Try it yourself (copy-paste)

Run these from the repo root. Works on Windows PowerShell or any shell with Python 3.

6.1 Run the vulnerable mock scan

```
cd C:\DevSecOps-Lab\secops-pipeline-lab
python ai_security_engineer_pack.py mock
```

You should see JSON with 53 findings, version 0.7.0-p7, and "pack_hands_complete": true.

6.2 Save output and run remediation

```
python ai_security_engineer_pack.py mock | Out-File -Encoding utf8 .tmp_se_vuln.json
python ai_remediation_engine.py .tmp_se_vuln.json
```

Look for mapped: 53 and unmapped: 0 in the remediation summary. A kit ZIP appears under hardening_kits/ (gitignored).

6.3 Run the clean mock (sanity check)

```
python ai_security_engineer_pack.py . mock_security_engineer_clean.json
```

Expect 0 findings - proves engines only fire when evidence exists.

Windows tip: Always use Out-File -Encoding utf8 when saving JSON. PowerShell's default > redirect uses UTF-16 and breaks json.load.

7. How this compares to other Sentinel role packs

Role pack	Tool ID	Engines	Mock findings (vuln)	Remediation IDs
Security Engineer	scan_security_engineer_pack	10	53	PERIM-* (53 mapped)
DevSecOps	scan_devsecops_pack	10	62	DEVSEC-*
Cloud Security	scan_cloud_pack	multi	170	CLOUD-*

All three share the same TOOL_STANDARDS JSON contract and the same Hands -> Brain -> Face roadmap.

8. What comes next (not in this report)

Step	Status	Plain English
Security Engineer Hands (P1-P6)	? Done	All 10 scanners work
Security Engineer Remediation (P7)	? Done	Every finding has a fix path
AI Security Engineer hands pack	? Next	Same pattern for AI/LLM security rol
Security Engineer Brain	Planned	AI agent orchestrates scans + triage
Face (GUI dashboard)	Planned	Visual ops console

There is no GUI yet. Output is JSON (+ remediation ZIP) until the Face phase.

9. One-page cheat sheet

```

Product ..... Security Engineer Multi-Engine Pack
Module ..... ai_security_engineer_pack.py
Version ..... 0.7.0-p7
Engines ..... 10 active / 0 stub
Mock vuln findings ... 53
Mock clean findings .. 0
ID scheme ..... PERIM-{CODE}-{NNN}
Remediation engine ... ai_remediation_engine.py v1.4.0
FIX_MAP PERIM entries 53 (0 unmapped)
Hands complete ..... YES (P6)
Remediation mapped ... YES (P7)
Next ..... AI Security hands -> Brain -> Face

```

10. Glossary for beginners

Term	Simple definition
Perimeter	The outer edge of your systems - net
Finding	One specific security problem with I
Engine	A focused checker inside the pack (e
Fixture / mock	Fake data file so you can demo witho
FIX_MAP	Lookup table: finding ID -> fix temp
Hardening kit	ZIP bundle of generated fix files fr
TOOL_STANDARDS	Shared JSON schema all Sentinel scan

Sentinel Stacks ? Security Engineer Hands P1-P7 ? Beginner build report